

[Java]

RFID UHF Reader API Instruction

Catalogs

RFID UHF Reader API Instruction	1
Release notes	4
1 API Summarize	5
1.1 API Construct	5
2 API Instruction	6
2.1 Supported languages and environments	6
2.2 API Basic flow of use	6
2.3 System Configuration Interface	7
2.3.1 Connect	7
2.3.2 Baud rate	10
2.3.3 Reader Address	10
2.3.4 Version Information	11
2.3.5 Beeper	11
2.3.6 Module Temperature	12
2.3.7 GPIO Status	12
2.4 RFID Interface Configuration	14
2.4.1 Antenna Connection Detection	14
2.4.2 Working Antenna	15
2.4.3 Output Power	16
2.4.4 Operating Frequency Region	18
2.4.5 RF link configuration	19
2.4.6 Return loss	22
2.4.7 Q Value	22
2.5 Label operation interface	24
2.5.1 Inventory Operation	24
2.5.2 Tag read/Write/Lock operation	27
2.5.3 Tag Configuration Filtering	27
2.6 Non-Standard tag	30
2.6.1 Monza tag	30
2.6.2 NXP	32
2.6.3 R6 lock	32
2.6.4 Johar thermometer tag	32
2.6.5 Fudan thermometer tag	32
2.7 Primary data structure[Java]	34

2.7.1 Connect the related data structures	34
2.7.2 System configuration related data structure	37
2.7.3 RF configuration enumerate	39
2.7.4 Tag operation related.....	40
3 Code specification	46
3.1 [Java] encoding specification	46
3.1.1 Naming style	46

Release notes

:

API_v[Interface protocol version].[[SDKVersion](#)]

[Interface protocol version](#): Module serial interface protocol version

[SDK Version](#): [Year].[Version No]

For instance:

Based on the V3.9 Interface protocol, the third version in 2020

So the final API version is set:

API_V4.0.2025.02

DATE	EDITION	DESCRIPTION
2020/6/10	V3.9.2020.3	Internal evaluation version
2025/02/07	V4.0.2025.02	Fix some api

1 API Summarize

1.1 API Construct

API Construct		
Application	Application	GUI demo program
Operating system	Operating system	Windows/Android/Linux
Interface	Library	Application Programming Interface: Supported Languages C++/c#/Java
Protocol	Communicate protocol	Communicate with hardware for data packaging and parsing
Hardware	Hardware	Product

2 API Instruction

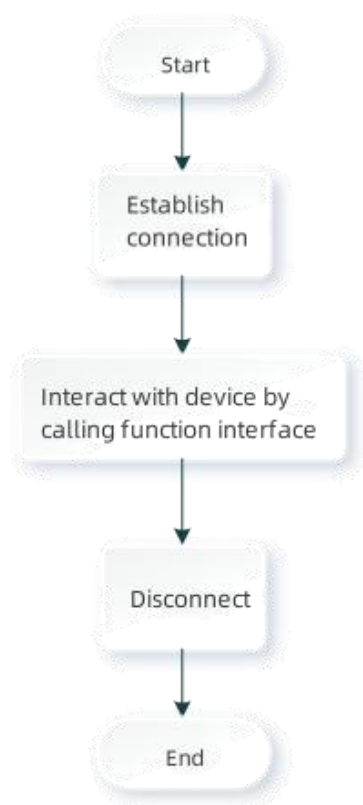
Common function interface for modules.

The function interface documentation describes how to connect modules and manipulate them.

2.1 Supported languages and environments

Platform	Java	C#	C++
Windows	Y	Y	Y
Android	Y	N/A	N/A
Linux	Y	N/A	Y

2.2 API Basic flow of use



Notes:

1, Code reference in addition to create, connect and disconnect, the reference code of the other interfaces are agreed Reader instance mReader has been created successfully, and the successful implementation of the connect () operation

2, Set the interface without special instructions will be saved to Flash, power outages are not lost!

2.3 System Configuration Interface

2.3.1 Connect

2.3.1.1 Creating a reader/writer object

Returns an instance of the class associated with the module on the particular communication channel. The communication channel is not established until the connect method is called.

Definition	Reader create(AntennaCount antennaCount)	
Parameter	antennaCount	Number of module antenna port (refer to class AntennaCount)
Return	Read-write instance of the specified device	
Identifying code	Reader mReader = null; mReader = Reader Impl .creat e (AntennaCount.SINGLE_CHANNEL);	

2.3.1.2 Connect

Open the communication channel and initialize the session with the reader/writer

Identifying code	bool connect(ConnectHandle handle)
Identifying code	ConnectHandle can be customized to achieve different connection mode, SDK has serial port, Bluetooth, network port
Identifying code	true – Connection Successful false – Connection failed
Identifying code	Reader mReader = null; bool connected = false; SerialPortHandle handle;

	<pre> mReader = ReaderImpl.create(AntennaCount.SINGLE_CHANNEL); handle = new SerialPortHandle("dev\\tty4", 115200);// Serial port connect handle = new NetworkHandle("IP",4001);// Network port connect // Bluetooth String bleAddress = bleAdapter.getData().get(position).getAddress(); RxBleHelper.getInstance().connect(bleAddress , connectState,connectSuccess, connectFail); // module connected connected = mReader.connect(handle); if (!connected) { // Connection failed, handing connection failure status } </pre>
--	---

2.3.1.3 Disconnect

.Disconnect with the reader.

Definiton	void disconnect()
Parameter	//
Return	//
Identifying code	<pre> Reader mReader = null; bool connected = false; // Create a handle to communicate with the module via Rs232. mReader = ReaderImpl.create(AntennaCount.SINGLE_CHANNEL); SerialPortHandle handle = new SerialPortHandle("dev\\tty4", 115200); // Real connection to the module connected = mReader.connect(handle); if (!connected) { // Connection failure, handling connection failure conditions } // Disconnect communicate with R/W if (true == mReader.disconnect()) { </pre>

	<pre> // Disconnect successful, release mReader object mReader = null; } else { // Handling the exception } </pre>
--	--

2.3.1.4 Getting Connected or not

definit on	void isConnected()	
paramet er		
return	void	
Referen ce code	mReader.isConnected();	

2.3.1.5 Reset Reader

Definiti on	void reset(Consumer<Failure> onError);	
Paramet er	onError	Failure to callback
Return	void	
Referen ce code	<pre> mReader.reset(new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures} }); </pre>	

2.3.1.6 Switch Antenna

Definiti on	void switchAntennaCount(AntennaCount antennaCount);	
Paramet er	antennaCount	enum: 1、4、8、16
Return	void	
Referen ce code	mReader.reset(AntennaCount.SINGLE_CHANNEL);	

2.3.2 Baud rate

2.3.2.1 Set Baud rate (Reserved and not recommended for use)

definition	void setBaudRate(BaudRate baudRate, Consumer<Success> onSuccess, Consumer<Failure> onError)	
parameter	baudRate onSuccess onError	Baud rate (Refer to enum BaudRate) Success Failed
return	void	
identifying code	<pre>mReader.setBaudRate(BaudRate._115200,new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } });</pre>	

2.3.3 Reader Address

2.3.3.1 Set Reader Address

Definition	void setReaderAddress(byte address,Consumer<Success> onSuccess,Consumer<Failure> onError)	
Parameter	address onSuccess onError	Reader Address, Value range0 – 254。 Successful Failed
Return	void	
Reference Code	<pre>mReader.setReaderAddress(0x01,new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Handling Successful } }, new Consumer<Failure>() { @Override</pre>	

	<pre> public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>
--	---

2.3.4 Version Information

2.3.4.1 Get Version information

Definiton	void getFirmwareVersion(Consumer<Version> onSuccess,Consumer<Failure> onError)
Parameter	onSuccess - Success onError - Failed
Return	void
Reference Code	<pre> mReader.getFirmwareVersion(new Consumer<Version>() { @Override public void accept(Version version) throws Exception { //Successful processing, version number and other information is in the Version }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>

2.3.5 Beeper

2.3.5.1 Set Beeper (Not recommended for the modules which are not connected with RodinBell development board. It will influence the reading speed.)

Definiton	void setBeeperMode(Beeper beeper, Consumer<Success> onSuccess, Consumer<Failure> onError)	
Parameter	beeper	Beeper mode
	onSuccess	Success
	onError	Failed
Return	void	

Referen ce Code	<pre> mReader.setBeeperMode(Beeper.QUIET, new Consumer<Version>() { @Override public void accept(Success success) throws Exception { //Successful Processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>
--------------------	--

2.3.6 Module Temperature

2.3.6.1 Query

Definiti on	void getReaderTemperature(Consumer<ReaderTemperature> onSuccess, Consumer<Failure> onError)
Paramet er	onSuccess - Successful onError - Failed
Return	void
Referen ce Code	<pre> mReader.getReaderTemperature(new Consumer<ReaderTemperature>() { @Override public void accept(ReaderTemperature temp) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>

2.3.7 GPIO Status

2.3.7.1 Read GPIO level

Definiti on	void readGpio(Consumer<GpioOut> onSuccess, Consumer<Failure> onError)
----------------	--

Parameter	onSuccess - Successful onError - Failed
Return	void
Reference code	<pre> mReader.readGpio(new Consumer<GpioOut>() { @Override public void accept(GpioOut gpio) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>

2.3.7.2 Set GPIO Level (Do not save to Flash)

Definiton	void writeGpio(GpioInType inType, boolean high, Consumer<Success> onSuccess, Consumer<Failure> onError)	
Parameter	inType high onSuccess onError	Gpio Status High level or not Success Failed
Return	void	
Reference code	<pre>mReader.writeGpio(new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful Processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } });</pre>	

2.4 RFID Interface Configuration

2.4.1 Antenna Connection Detection

2.4.1.1 Set Antenna Connection Detector

Definiton	void setAntConnectionDetector(byte detectorSensitivity, Consumer<Success> onSuccess, Consumer<Failure> onError)	
Parameter	detectorSensitivity onSuccess onError	0 : Turn off the antenna connection detection. >0: Sensitivity of antenna connection detection (port echo loss value) in dB. The larger the value is, the higher the impedance matching requirement of the port is. success Failed
Return	void	
Reference code	<pre>mReader.setAntConnectionDetector(0x00, new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } });</pre>	

2.4.1.2 Get Antenna Connection Detector

Definiton	void getAntConnectionDetector(Consumer<AntConnectionDetector> onSuccess, Consumer<Failure> onError)	
Parameter	onSuccess - Success onError - failed	

Return	void
Reference code	<pre> mReader.getAntConnectionDetector(0x00, new Consumer<AntConnectionDetector>() { @Override public void accept(AntConnectionDetector detector) throws Exception { //Successful Processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>

2.4.2 Working Antenna

2.4.2.1 Set Working Antenna (Not save to Flash)

Definition	void setWorkAntenna(int antennaId, Consumer<Success> onSuccess, Consumer<Failure> onError)	
Parameter	antennaId onSuccess onError	To set the antenna number as the working antenna success Failed
Return	void	
Identifying code	<pre>mReader.setWorkAntenna(0x01, new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } });</pre>	

2.4.2.2 Query working antenna of the reader

2.4.2.3

Definiton	void getWorkAntenna(Consumer<WorkAntenna> onSuccess, Consumer<Failure> onError)
Parameter	onSuccess - Success onError - failed
Return	void
Identifying code	<pre>mReader.getWorkAntenna(new Consumer<WorkAntenna>() { @Override public void accept(WorkAntenna antenna) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } });</pre>

2.4.3 Output Power

2.4.3.1 Set RF output power of the reader

Definiton	void setOutputPowerUniformly(byte uniformPower, boolean isTemporary, Consumer<Success> onSuccess, Consumer<Failure> onError)	
Parameter	uniformPower isTemporary onSuccess onError	Power of all of the antennas RF output power, value range 0-33 (0x00 0x21) in dBm. Temporary setting not success Failed
Return	void	

Identifying code	<pre> mReader.setOutputPowerUniformly(0x33, True, new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>
------------------	--

2.4.3.2 Set the RF output power for each antenna

2.4.3.3

Definition	<pre> void setOutputPower(OutputPowerConfig powerConfig, Consumer<Success> onSuccess, Consumer<Failure> onError) </pre>	
Parameter	<pre> powerConfig onSuccess onError </pre>	Antenna power parameter configuration class success Failed
Return	void	
Identifying code	<pre> mReader.setOutputPower(OutputPowerConfig.outputPower(...), new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful Processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>	

2.4.3.4 Query output power of the reader (getOutputPower)

Definition	void getOutputPower(Consumer<OutputPower> onSuccess, Consumer<Failure> onError)	
Parameter	onSuccess - Success onError - Failed	
Return	void	
Identifying code	<pre> mReader.getOutputPower(new Consumer<OutputPower>() { @Override public void accept(OutputPower success) throws Exception { //successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>	

2.4.4 Operating Frequency Region

2.4.4.1 Set operating frequency region of the reader

Definition	void setFrequencyRegion(FreqNormal freqNormal, Consumer<Success> onSuccess, Consumer<Failure> onError)	
Parameter	freqNormal onSuccess onError	Operating frequency region configuration class success Failed
Return	Void	
Identifying code	<pre> mReader.setFrequencyRegion(new FreqNormal.Builder().build(), new Consumer<Success>() { @Override </pre>	

	<pre> public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>
--	---

2.4.4.2 Query reader' s operating region

Definit on	void getFrequencyRegion(Consumer<FreqRegionResult> onSuccess, Consumer<Failure> onError)
Paramet er	onSuccess - Successful onError - failed
Return	void
Identify ing code	<pre> mReader.getFrequencyRegion(new Consumer<FreqRegionResult>() { @Override public void accept(FreqRegionResult success) throws Exception { //successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>

2.4.5 RF link configuration

2.4.5.1 Set RF link configuration

Definit on	void setRfLinkProfile(ProfileId profileId, Consumer<Success> onSuccess, Consumer<Failure> onError)	
Paramet er	profileId onSuccess onError	Communication rate of the RF link (refer to enu m profileId) success Failed

Return	void
Identifying code	<pre> mReader.setRfLinkProfile(ProfileId .Profile0, new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>

2.4.5.2 Read RF link configuration

Definition	void getRfLinkProfile(Consumer<RfLinkProfile> onSuccess, Consumer<Failure> onError)
Parameter	onSuccess-Success OnError-Failure
Return	void
Identifying code	<pre> m Reader .getRfLinkProfile(new Consumer<RfLinkProfile>() { @Override public void accept(RfLinkProfile success) throws Exception { // Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { // Processing failures } }); </pre>

2.4.5.3 Get the link for the E710

Definiton	void getE710LinkProfile(Consumer<E710LinkProfile> onSuccess, Consumer<Failure> onError)
Parameter	onSuccess-Success OnError-Failure
Return	void
Identifying code	<pre> m Reader .getE710LinkProfile(new Consumer<E710LinkProfile>() { @Override public void accept(E710LinkProfile success) throws Exception { // Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { // Processing failures} }); </pre>

2.4.5.4 Get the link for the E710

definiton	void getE710LinkProfile(Consumer<E710LinkProfile> onSuccess, Consumer<Failure> onError)
parameter	onSuccess-Success OnError-Failure
return	void
identifying code	<pre> m Reader .getE710LinkProfile(new Consumer<E710LinkProfile>() { @Override public void accept(E710LinkProfile success) throws Exception { // Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>

2.4.6 Return loss

2.4.6.1 Query the Return loss of the antenna port

The system will obtain the return loss value for operating antenna port at the specified frequency point.

definition	void getRfPortReturnLoss(byte frequency, Consumer<RfPortReturnLoss> onSuccess, Consumer<Failure> onError)	
parameter	Frequency onSuccess onError	For frequency parameters, please refer to the corresponding table for frequency parameters. success failed
return	void	
identifying code	<pre> m Reader .getRfPortReturnLoss(0x00, new Consumer<RfPortReturnLoss>() { @Override public void accept(RfPortReturnLoss success) throws Exception { // Successful processing, return loss value, in dB. $VSWR = (10^{RL/20} + 1) / (10^{RL/20} - 1)$。 } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { // Processing failures } }); </pre>	

2.4.7 Q Value

2.4.7.1 Set E710 Q Value

Definition	void setE710_Q(QInfo qInfo, Consumer<Success> onSuccess, Consumer<Failure> onError);	
Parameter	qInfo onSuccess onError	configuration parameter success Failed
Return	void	

Identifying code	<pre> m Reader .setE710_Q(qInfo, new Consumer<Success>() { @Override public void accept(Success success) throws Exception { promptUi(getString(R.string.set_success)); } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { promptUi(getString(R.string.set_error)); } }); </pre>			
qInfo Instruction	QMode	Q mode	FIXED	Fixed Q mode
			DYNAMIC	Dynamic Q mode
	QInit	Start Q value		0~15, and is somewhere between the QMin and the QMax
	QMax	Maximum Q value		0~15, and is greater than or equal to QMin
	QMin	Minimum Q value		0~15, and is less than or equal to QMax
	NumMinQ Cycles	No valid EPC at the minimum Q-value, Query / QueryAdj attempts		Greater than or equal to 0, the default is 1
	MaxQuerySinceEPC	Maximum Query / QueryAdj times allowed after receiving a valid EPC		Greater than or equal to 0, the default is 0

2.4.7.2 Read E710Q Value

Definition	void getE710_Q(Consumer<QInfo> onSuccess, Consumer<Failure> onError);	
Parameter	onSuccess onError	Success Failed
Return	void	
Identifying code	<pre> mReader.getE710_Q(new Consumer<QInfo>() { @Override public void accept(QInfo qInfo) throws Exception { } } </pre>	

	<pre> }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { } }); </pre>
--	---

2.5 Label operation interface

2.5.1 Inventory Operation

2.5.1.1 Mode configuration

2.5.1.1.1 Real-time inventory mode configuration

Definiton	<pre> new CustomSessionTargetInventory.Builder() .session(Session session) .target(Target target) .selectFlag(SelectFlag selectFlag) .enablePhase(boolean enablePhase) .powerSave(byte powerSave) .repeat(byte repeat) .build(); </pre>	
Parameter	session	Specify inventory session (refer to enum Session)
	target	Specify inventory Inventoried Flag (refer to enum Target)
	enablePhase	Read phase value true – Read phase value false – Do not read phase value
	repeat	Number of repetitions of inventory process
	selectFlag	Select Flag (Refer to enum SelectFlag)
	powerSave	Delay before the command return 2ms / 1bit.(Default: 100)
Return	Parameter entity class	

Identifying code	
------------------	--

2.5.1.1.2 Fast inventory mode configuration

Definition	<pre> new FastSwitchSingleAntennaInventory.Builder() .interval(byte interval) .session(Session session) .target(Target target) .repeat(byte repeat) .optimize(byte optimize) .targetQuantity(byte targetQuantity) .ongoing(byte ongoing) build() </pre>	
Parameter	session	Specify session to be inventoried (Refer to enum Session)
	target	Specify target of Inventoried Flag (Refer to enum Target)
	enablePhase	Read phase value true – Read phase value false – Do not read phase value
	repeat	Number of repetitions of inventory process
	antennaStay	Polled antenna and repeat number table (refer to module specification for setting)
	interval	Rest time between the antennas polling. The unit is mS. No RF output at rest, which can reduce the power consumption.
	optimize	Optimize the parameters
	ongoing	continuation
	targetQuantity	target Quantity
	tempPower	Temporary power of antenna
Return	Parameter entity class	
Identifying code		

2.5.1.2 Inventory

2.5.1.2.1 Start inventory

Definiton	void setInventoryConfig(InventoryConfig config)
Parameter	config - Parameter configuration class
Return	void
Identifying code	<pre>InventoryConfig config = new InventoryConfig.Builder() .setInventory(CustomSessionTargetInventory inventory) .setOnInventoryTagSuccess(Consumer<InventoryTag> onSuccess) .setOnInventoryTagEndSuccess(Consumer<InventoryTagEnd> onSuccess) .setOnFailure(Consumer<InventoryFailure> onFailure) .build(); //Set inventory parameter mReader.setInventoryConfig(config); //Start inventory, Parameter: Fast inventory or not mReader.startInventory(true); //Stop inventory mReader.stopInventory();</pre>

2.5.1.2.2 Stop inventory

Definiton	void stopInventory();
Parameter	Null
Return	void
identifying code	(Refer to Start inventory)

2.5.2 Tag read/Write/Lock operation

Definition	<pre>//Read tag void readTag(ReadConfig config, Consumer<OperationTag> onSuccess, Consumer<Failure> onError) //Read tag void readTag(CustomSessionReadConfig config, Consumer<OperationTag> onSuccess, Consumer<Failure> onError) //Write tag void writeTag(WriteConfig config, boolean blockWrite, Consumer<OperationTag> onSuccess, Consumer<Failure> onError) //Lock tag void lockTag(LockConfig config, Consumer<OperationTag> onSuccess, Consumer<Failure> onError) //Kill tag void killTag(KillConfig config, Consumer<OperationTag> onSuccess, Consumer<Failure> onError)</pre>	
Parameter		
Return	void	
Identifying code		

2.5.3 Tag Configuration Filtering

2.5.3.1 Set EPC match configuration

Definition	<pre>void setEpcMatch(MatchConfig config, Consumer<Success> onSuccess, Consumer<Failure> onError);</pre>	
Parameter	无	

Return	void
Identifying code	<pre>String noSpacesEpc = epcStr.replace(" ", ""); MatchConfig config = new MatchConfig.Builder() .setMaskValue(noSpacesEpc) .build(); mReader.setEpcMatch(config, new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } });</pre>

2.5.3.2 Get EPC match configuration

Definition	void getEpcMatch(Consumer<MatchInfo> onSuccess, Consumer<Failure> onError);
Parameter	Null
Return	void
Identifying code	<pre>mReader.getEpcMatch(new Consumer<MatchInfo>() { @Override public void accept(MatchInfo success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } });</pre>

2.5.3.3 Clear EPC match configuration

Definition	void clearEpcMatch(Consumer<MatchInfo> onSuccess, Consumer<Failure> onError);
Parameter	Null
Return	void

Identifying code	<pre> mReader.clearEpcMatch(new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failures } }); </pre>
------------------	---

2.5.3.4 Set tag filtering

Definition	void setTagMask(MaskConfig config, Consumer<Success> onSuccess, Consumer<Failure> onError);
Parameter	无
Return	void
Identifying code	<pre> mReader.setTagMask(new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failure } }); </pre>

2.5.3.5 Get tag filtering

Definition	void getTagMask(Consumer<Success> onSuccess, Consumer<Failure> onError);
Parameter	无
Return	void
Identifying code	<pre> mReader.getTagMask(new Consumer<MaskInfo>() { @Override public void accept(Success success) throws Exception { //Successful processing } }); </pre>

	<pre> } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //processing Failure } }); </pre>
--	---

2.6 Non-Standard tag

2.6.1 Monza tag

2.6.1.1 Set Monza tag fast read TID (Optional save to Flash)

Definition	<pre> void setImpinjFastTid(FastTidType fastType, boolean save, Consumer<Success> onSuccess, Consumer<Failure> onError) </pre>	
Parameter	fastType	Enum class
	Save	Save to flash or not
	onSuccess	Success
	onError	Failed
Return	void	
Identifying code	<pre> mReader.setImpinjFastTid(FastTidType.DISABLE, false, new Consumer<Success>() { @Override public void accept(Success success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failure } }); </pre>	

2.6.1.2 Get fast TID setting

Definiton	void getImpinjFastTid(Consumer<ImpinjFastTid> onSuccess, Consumer<Failure> onError)
Parameter	onSuccess - Success onError - Failed
Return	void
Identifying code	<pre>mReader.getImpinjFastTid(new Consumer<ImpinjFastTid>() { @Override public void accept(ImpinjFastTid success) throws Exception { //Successful processing } }, new Consumer<Failure>() { @Override public void accept(Failure failure) throws Exception { //Processing failure } });</pre>

2.6.2 NXP

2.6.2.1 Change EAS

2.6.2.2 Untraceable

2.6.3 R6 lock

2.6.3.1 cmd_lock

2.6.4 Johar thermometer tag

2.6.4.1 Enable Johar thermometer tag

2.6.4.2 Disable Johar thermometer tag

2.6.5 Fudan thermometer tag

2.6.5.1 Thermometric

2.6.5.1.1 Single temperature measurement of Meas

2.6.5.1.2 Open RTC temperature measurement

2.6.5.1.3 Stop RTC temperature measurement

2.6.5.2 Register

2.6.5.2.1 Load Configuration load reg

2.6.5.2.2 Read register read reg

2.6.5.3 Storage

2.6.5.3.1 read memory

2.6.5.3.2 Write memory

2.6.5.4 State check state check

2.6.5.5 authenticate auth

2.6.5.6 Led Control led

2.7 Primary data structure[Java]

2.7.1 Connect the related data structures

2.7.1.1[Enum] BaudRate

Baud rate	<pre>/** * Baud rate */ enum BaudRate { /** * 38400 bps */ _38400(0x03), /** * 115200 bps */ _115200(0x04); }</pre>
-----------	--

2.7.1.2 [JAVA INTERFACE] ConnectHandle

ConnectHandle	<pre>/** * Connect handle */ interface ConnectHandle { /** * Connected devices * * @return Whether the connection is successful */ boolean onConnect(); /** * Is it connected * * @return Is it connected */ }</pre>
---------------	---

	<pre> */ boolean isConnected(); /** * Send data to device * * @param data byte[] */ void onSend(byte[] data); /** * Callback data received from device * * @param consumer {@link Consumer} */ void onReceive(Consumer<byte[]> consumer); /** * Disconnect device */ void onDisconnect(); } </pre>
--	---

2.7.1.3 **[java implement class] connect handle** : Rs232 connect handle

SerialPortHandle

Connect handle : Rs232 connect handle implementation class	<pre> public class SerialPortHandle implements ConnectHandle { private String mPort; private int mBaud; public SerialPortHandle(String port, int baud) { this.mPort = port; this.mBaud = baud; } @Override public boolean onConnect() { //Implement } @Override public boolean isConnected() { //Implement } } </pre>
--	--

	<pre> } @Override public void onSend(byte[] bytes) { //Implement } @Override public void onReceive(Consumer<byte[]> consumer) { //Implement } @Override public void onDisconnect() { //Implement } } </pre>
--	--

2.7.1.4 [java Implement Class] Tag operation : Read tag data area

ReadData

Tag operation : read tag data area	<pre> /** * tag operation : read tag data area*/ class ReadData extends TagOp { /** * tag data area */ MemBank memBank; /** * Start word address of the data area to be read */ int wordAddress; /** * Length of data area to be read, Unit word */ int wordLen; /** </pre>
------------------------------------	---

	<pre> * Default constructor */ ReadData() { } /** * Read the data area with parameters constructor * * memBank Data area to be read * wordAddress Starting address of the data area to be read * wordLen length of the data area to be read, Unit word */ ReadData(MemBank memBank, int wordAddress, int wordLen) { this.memBank = memBank; this.wordAddress = wordAddress; this.wordLen = wordLen } } </pre>
--	--

2.7.2 System configuration related data structure

2.7.2.1 [Enum] Status of the beeper when operating the tag **BeeperMode**

Status of the beeper when operating the tag	<pre> /** * beeper mode when operating the tag */ enum BeeperMode { /** * Quiet */ Quiet(0x0), /** * Buzzer after each inventory </pre>
---	---

	<pre> */ BuzzerAfterInventory(0x1), /** * Buzzer every time read a tag */ BuzzerFoundTag(0x2); } </pre>
--	---

2.7.2.2 [enum] GPIO state Gpio Pin

GPIO state	<pre> /** * GPIO state */ class GpioPin { /** * id No */ int id; /** * Level * true - High * false - Low */ boolean high; /** * GPIO direction * true - output * false - input */ boolean output; } </pre>
------------	--

2.7.2.3 [enum] version information

Version info	<pre> /** * version info */ class VersionInfo { /** </pre>
--------------	--

	<pre> * Firmware major version no */ String major; /** * Firmware minor version no */ String minor; /** * 12 bytes reader identification No */ String identifier; } </pre>
--	--

2.7.3 RF configuration enumerate

2.7.3.1 [enum] Communication rate of RF link ProfileID

communicate rate of RF link	<pre> /** * communicate rate of RF link * TODO: verify the name has a better */ enum ProfileID { /** * Tari 25uS, FM0 40KHz */ Profile0(0xD0), /** * Tari 25uS, Miller 4 250KHz */ Profile1(0xD1), /** * Tari 25uS, Miller 4 300KHz */ Profile2(0xD2), /** * Tari 6.25uS, FM0 400KHz */ Profile3(0xD3); } </pre>
-----------------------------	--

2.7.3.2 [Java class] FrequencyRegion

Frequency Region	<pre>/** * operating frequency region * Set the range of frequency hopping within the frequency range of RF * specification.Rules of parameter: * 1. The start frequency and end frequency must not exceed the range of * the RF specification. * 2. The start frequency must be lower than the end frequency. * 3. The start frequency is equal to the end frequency for fixed frequency * transmission. */ class FrequencyRegion { /** * System default frequency point, frequency band */ Region region; /** * frequency onset point */ int startFreq; /** * frequency end point */ int endFreq; }</pre>
------------------	--

2.7.4 Tag operation related

2.7.4.1 [enum] Frequency Region

Frequency range	<pre>/** * Frequency range */ enum Region { /** * RF specification for FCC */ }</pre>
-----------------	---

	<pre> FCC(0x0), /** * RF specification for ETSI */ ETSI(0x1), /** * RF specification for CHN */ CHN(0x2), /** * Customized RF specification */ CUSTOM(0x4); } </pre>
--	---

2.7.4.2 [Enum] Tag data area MemBank

Tag data area	<pre> /** * Tag data area */ enum MemBank { /** * Reserved */ RESERVED (0x0), /** * Epc */ EPC (0x1), /** * Tid */ TID (0x2), /** * User */ USER (0x3); } </pre>
---------------	--

2.7.4.3 [Enum] Session

Session	<pre>/** * Session */ enum Session { /** * Session 0 */ S0(0x0), /** * Session 1 */ S1(0x1), /** * Session 2 */ S2(0x2), /** * Session 3 */ S3(0x3); }</pre>
---------	--

2.7.4.4 [enum] Inventory target Target

Inventory target	<pre>/** * Inventory target */ enum Target { /** * Inventoried Flag A */ A(0x0), /** * Inventoried Flag B */ B(0x1); }</pre>
------------------	--

2.7.4.5 [Enum] Tag lock type LockType

tag lock type	<pre>/** * tag lock type */ enum LockType { /** * unlock */ UNLOCK(0x0), /** * lock */ LOCK(0x1), /** * permanent lock */ PERMAUNLOCK(0x2), /** * permanent lock */ PERMALOCK(0x3), /** * permanent lock R6 tag */ PERMALOCK_R6(0x6); }</pre>
---------------	---

2.7.4.6 [javacallback] setOriginalDataCallback 、
addOriginalDataReceivedCallback

setOriginalDataCallback	<pre>/** * set up callback methods that monitoring original command data *Interested in processing original command data can be registered with tReader.setOriginalDataCallback(). */ mReader.setOriginalDataCallback(new Consumer<byte[]>() { @Override public void accept(byte[] onSend) throws Exception {</pre>
-------------------------	--

	<pre> //Original command data sends callback } }, new Consumer<byte[]>() { @Override public void accept(byte[] onReceive) throws Exception { //Original command data receives callback } }); </pre>
--	---

2.7.4.7 [java interface] tag operation TagOp

TagOp	<pre> /** *TagOp */ interface TagOp { } </pre>
-------	--

2.7.4.8 [Java Class] TagReadData

Tag read data	<pre> /** * Tag read data */ class TagReadData { /** * pc String */ String pc; /** * epcString */ String epc; /** * Frequency */ int frequency /** * Antenna no </pre>
---------------	---

	<pre> */ int antenna /** * RSSI */ int rssi; /** * phase value */ int phase } </pre>
--	--

2.7.4.9 [Java Class] TagOpResult

Tag Operation result	<pre> /** * Tag Op Result */ class TagOpResult { /** * Total number of successfully operated tags */ int tagCount; /** * The effective data length of the manipulated tag. unit is bytes. */ int dataLen; /** * Valid data for the operated tag */ byte[] data; /** * Operation result * read Operation is the length of the read, Unit is Byte */ int result; /** * frequency parameter */ int frequency; /** * Antenna No </pre>
----------------------	--

	<pre> */ int antennaNo; /** * Number of times the tag has been successfully manipulated */ int tagOpCount; } </pre>
--	---

3 Code specification

3.1 [Java] encoding specification

3.1.1 Naming style

3.1.1 Naming style

- (a) Use the UpperCamelCase style for class names. Except for RFID proper names.
- (b) Method names, parameter names, member variables, and local variables all use the lowerCamelCase style.
- (c) Constants should be named in all capitals, separated by underscores, and the semantics should be clear and complete, so don't mind the length of the name.
- (d) Abstract class naming starts with Abstract; Exception class naming ends with Exception; Test class naming starts with the name of the class it wants to test and ends with Test.
- (e) Types are enclosed in parentheses to represent arrays.
Positive example: `int [] arrAntenna`
Reverse example: `int [][] arrGpio`
- (vi) Package names are uniformly lowercase, with one and only one English word of natural semantics between the dot separators. Package names are uniformly used in singular form, but class names may be used in plural form if they have a plural meaning.
- (vii) Avoid identical naming between member variables of child and parent classes, or between local variables in different code blocks, making readability less likely.
- (viii) Eliminate completely unstandardized abbreviations, to avoid looking at the text without knowing the meaning.
- (ix) Do not add any modifiers to methods and properties in interface classes (not even public), keep the code

simple, and add valid Javadoc comments. Try not to define variables in interfaces. If you must define variables, make sure they are related to the interface method and are constants that are fundamental to the entire application.

3.1.2 Constant Definitions

(a) When assigning a value to a long or a long, use an uppercase L after the value, not a lowercase l. Lowercase is easily confused with numeric, causing misunderstandings.

(b) If the value of a variable changes only within a fixed range, use the enum type to define it.

3.1.3 Code Format

(i) If it is empty within the curly brackets, write it succinctly as `{ }` without line breaks and spaces in between the curly brackets; if it is a non-empty code block then:

No line break before the left curly bracket.

Line break after left curly brace.

The right curly brace is preceded by a line break.

If there is any code such as `else` after the right curly bracket, there is no line break;

The right curly bracket must be followed by a new line to indicate termination.

(ii) left parentheses and the right side of the neighboring characters do not appear between the space; right parentheses and the left side of the neighboring characters do not appear between the space; and the left curly brackets before the need to add a space.

(iii) Spaces are required between reserved words such as `if/for/while/switch/do` and parentheses.

(d) A space is required on the left and right sides of any binomial or trinomial operator.

(e) Use four spaces for indentation, and the tab character is prohibited.

(vi) There should be one and only one space between the double slash of a comment and the content of the comment.

(vii) In the case of type-forced conversions, no space is required between the right bracket and the forced-conversion value.

(viii) The number of characters in a single line is limited to no more than 120, beyond the need for line breaks, line breaks in accordance with the following principles:

The second line is indented 4 spaces relative to the first line, starting from the third line, no further indentation.

Operators are linefiled with the following.

Dot notation for method calls is included with the following line breaks.

When multiple arguments in a method call require a line break, do so after a comma.

Do not make line breaks before parentheses

(ix) Method parameters must be followed by a space after the comma for multiple parameters when defined and passed in.

(j) IDE's text file encoding is set to UTF-8; use Unix format for line breaks in IDE files, do not use Windows

format.

3.1.4 Object-oriented conventions

(i) Avoid accessing static variables or static methods of a class through an object reference of the class, which needlessly increases the compiler's parsing cost, and access them directly with the class name.

(ii) All override methods must be annotated `@Override`.

(iii) Obsolete classes or methods must not be used.

(d) The equals method of Object is prone to throwing null pointer exceptions, so equals should be called with a constant or an object that is certain to have a value.

(e) All integer wrapper class objects between the value of the comparison, all use equals method comparison.

(F) floating-point numbers between the equal value judgment, the basic data types can not be used `==` to compare, packaged data types can not be used equals to determine. Explanation: Floating-point numbers are encoded in the form of "mantissa + ordinate", which is similar to the representation of "significant digit + exponent" in scientific notation. Binary cannot accurately represent most decimal fractions.

(vii) It is forbidden to add any business logic inside the constructor method. If there is initialization logic, please put it in the init method.

(viii) When a class has more than one constructor method, or more than one method with the same name, these methods should be placed together in order to make them easier to read.

(ix) In the setter method, the parameter name is the same as the class member variable name, `this.member name = parameter name`.

(j) In getter/setter methods, do not add business logic, making it more difficult to troubleshoot problems.

(xi) within the loop, the string connection, use the `StringBuilder` append method to extend.

(xii) final can declare classes, member variables, methods, and local variables, and the final keyword is used in the following cases:

Classes that are not allowed to be inherited, such as: `String` class.

Domain objects that are not allowed to modify references, such as domain variables of the POJO class.

Methods that are not allowed to be overridden, such as the setter method of the POJO class.

Local variables that are not allowed to be reassigned during runtime.

Avoid contextual reuse of a variable, use `final` to force redefinition of a variable for better refactoring.

(xiii) Class member and method access control is strict:

Constructor methods must be private if they do not allow objects to be created directly from the outside by new.

Tool classes are not allowed to have public or default constructors.

Classes with non-static member variables that are shared with subclasses must be protected.

Class non-static member variables that are used only in this class must be private.

A class static member variable must be private if it is used only in this class, and final if it is a static member variable.

Class member methods are for internal use only and must be private.

A class member method is restricted to being protected if it is public only to the inheriting class.

Note: Any class, method, parameter, or variable is strictly scoped. An overly broad access scope is not conducive to module decoupling.

3.1.5 Date and Time

(a) When formatting a date, pass in pattern to indicate the year in lower case y.

(b) date formatting, yyyy said the day in the year

(c) In the date format, make clear the significance of the upper-case M and lower-case m, and the upper-case H and lower-case h, respectively.

For months, it is capital M. For minutes, it is lowercase m;

Minutes are represented by a lowercase m;

Capital H for a 24-hour day;

For 12-hour system, it is lowercase h.

(d) to get the current milliseconds: `System.currentTimeMillis();` (e) do not get the current milliseconds.

(v) Not allowed anywhere in the program:

`java.sql.Date`

`java.sql.Time`

`java.sql.Timestamp.`

3.1.6 Collection processing

(i) Determine whether all elements inside a collection are empty, using the `isEmpty()` method, rather than the `size()==0` approach.

3.1.7 Control statements

(i) Within a switch block, each case is either terminated by `continue/break/return`, etc., or annotated to indicate which case the program will continue to execute to; a default statement must be included at the end of a switch block, even if it contains no code.

Note: Note that `break` exits the switch block and `return` exits the method body.

When the variable inside the switch brackets is of type `String` and the variable is an external parameter, a null judgment must be performed first.

Curly braces must be used in `if/else/for/while/do` statements.

3.1.8 Annotation Conventions

(a) Classes, class attributes and class methods must be annotated using the Javadoc specification, using the `/**content**/` format, and must not use the `// xxx` method.

(b) All abstract methods (including methods in interfaces) must be annotated with Javadoc, in addition to the return value, parameters, exceptions, it must also point out what the method does and what it accomplishes. Note: The

implementation requirements of the subclasses, or call notes, please also explain.

(iii) All classes must have a creator and creation date added.

(iv) Note: When setting up templates, note that IDEA's @author is ``${USER}``, while eclipse's @author is ``${user}``, with case differences, and the date is set uniformly in the format yyyy/MM/dd.

(e) method within a single line comment, in the commented statement above another line, use `//` comment. Methods within the multi-line comments using `/* */` comments, pay attention to alignment with the code.

(vi) All enumerated type fields must be commented to explain the purpose of each data item.

3.1.9 Multithreading Conventions

(i) In the use of blocking wait to acquire a lock, it must be outside the try code block, and there must not be any method call between the locking method and the try code block that may throw an exception, to avoid being unable to unlock the lock in finally after the lock has been successfully added.

(Note 1: If the method call between the lock method and the try block throws an exception, then the lock cannot be unlocked and other threads cannot successfully acquire the lock.

(iii) Explanation 2: If the lock method is within the try block, an exception may be thrown by another method, resulting in an unlock in the finally block.

(iv) To unlock an unlocked object, it calls the AQS tryRelease method (depending on the implementation class), throwing an `IllegalMonitorStateException`.

(Note 3: An unchecked exception may be thrown in the implementation of the lock method of a Lock object, with the same consequences as in Note 2.

3.2 Exception Handling Conventions

(a) `RuntimeException` exceptions defined in the Java class library that can be circumvented by pre-checking should not be handled by catch, for example: `NullPointerException`, `IndexOutOfBoundsException`, etc.

Note: Exceptions that cannot be pre-checked, such as parsing numbers in string form, may have a number formatting error and have to be caught by catching `NumberFormatException`.

Example: `if (obj != null) {...}`

Counterexample: `try { obj.method(); } catch (NullPointerException e) {...}`

(ii) Exceptions should not be used for flow control, conditional control.

(iii) When catching, please distinguish between stable code and non-stable code. Stable code is code that will not go wrong no matter what. Stable code is code that will not go wrong no matter what. For non-stable code catch, try to differentiate the type of exception, and then do the corresponding exception handling.

(d) Catch an exception in order to deal with it, don't catch it and discard it without dealing with anything, if you

don't want to deal with it, throw the exception to its caller. The outermost business user, must handle the exception and convert it into something the user can understand.

(v) Finally blocks must close on resource objects, stream objects, and do a try-catch if there is an exception.

(vi) Do not use return in finally blocks.

(vii) Catch exceptions and throw exceptions must be an exact match, or the catch exception is the parent class of the throw exception.